

RELATÓRIO EXPERIMENTO 5

Nome: Renato Gabriel Lima da Silva

Matrícula: 251027871

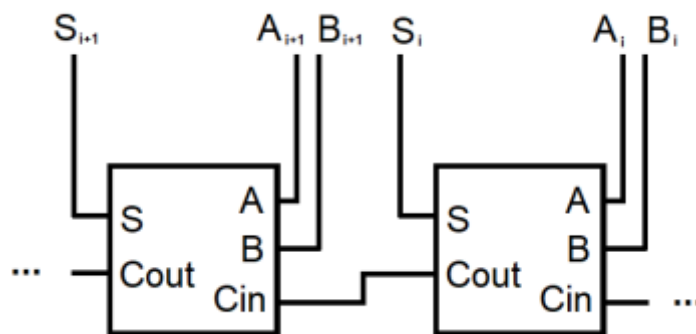
INTRODUÇÃO

Neste experimento, iremos abordar um somador de palavras binárias de 4 bits em VHDL e simular no software ModelSim, sendo um feito com um Somador completo em cascata, e outro feito por meio de um operador “+” da biblioteca STD_LOGIC_ARITH e por último, fazer um testbench para simular e testar todas as combinações possíveis de valores A e B e comparar as saídas dos dois somadores. Vamos, portanto, abordar seus conceitos e suas funcionalidades.

TEORIA

1 - Questão 1

O primeiro exercício consiste na modelagem do DUT, um somador de palavras de 4 bits sem overflow, ou seja, que possui uma saída de 5 bits. Para isso, devem ser implementadas duas entradas, A e B, ambas de 4 bits, e uma saída S de 5 bits. As entradas serão as palavras a serem somadas, enquanto a saída será o resultado da soma. E isso deve ser feito utilizando o somador completo, feito em experimentos anteriores em cascata.



- Figura 1: Modelo de cascadeamento de somadores completos

Com esse modelo conseguimos seguir a lógica juntando 4 desses somadores obtemos a soma S, é importante ressaltar que o primeiro Cin recebe a entrada 0 e o último Cout vai ser o nosso S5.

2 - Questão 2

O exercício 2 consiste na modelagem do GM, um somador de palavras que garantidamente sempre providenciará o resultado correto. O Golden Model será utilizado para determinar se o DUT, produzido no exercício anterior, apresenta a funcionalidade desejada. A fim de garantir a construção de um Golden Model perfeitamente funcional, usa-se da biblioteca padrão STD_LOGIC_ARITH, que inclui o operador +. Com ele, é possível realizar somas com números em decimais, e então convertê-los para binários. Dito isso, a montagem do GM é demasiadamente simples. Basta aceitar duas entradas, as palavras de 4 bits A e B, concatená-las à esquerda com uma constante 0, e, enfim, somá-las. O resultado será a saída S de 5 bits

3 - Questão 3

O último exercício, exercício 3, consiste na construção do testbench que irá coordenar as entradas no DUT e GM, e comparar ambas as saídas. O Testbench apresenta uma engenharia mais complexa. Como será realizado o teste de um somador de palavras de 4 bits, existem 16 diferentes possibilidades para cada palavra, o intervalo de inteiros de 0 até 15. São duas entradas de palavras, então serão 256 testes no total. Para realizar todas as possíveis combinações para a palavra A e a palavra B, faz-se dois loops, um interno ao outro, a fim de obter todas as 256 combinações. Em cada uma, será checado se o valor obtido pelo DUT é equivalente ao do GM. Se não, o DUT está incorretamente implementado

CÓDIGOS

1- Somador Completo

Neste experimento utilizamos a linguagem de VHDL por meio do software Modelsim para desenvolver o Somador, entretanto, para fazer o dut precisamos anteriormente do código do somador completo, portanto segue o mesmo

```
1      library IEEE;
2      use IEEE.STD_LOGIC_1164.all;
3
4      entity somador is
5          port(A,B,Cin: in std_logic;
6              S, Cout: out std_logic);
7      end somador;
8
9      architecture somador_arch of somador is
10     begin
11         S <= A xor B xor Cin;
12         Cout <= (A and B) or (A and Cin) or (B and Cin);
13
14     end somador_arch;
```

- Listagem 1: Código do somador completo

Nas primeiras e segundas linhas definimos as bibliotecas que serão usadas no código no caso a “IEEE.STD_LOGIC_1164.all”. Em seguida, nas linhas 4 a 7, temos a definição de entidade do somador definindo as variáveis de entrada (A,B e Cin) e as de saídas (S e Cout). Depois nas linhas 9 a 14 temos por fim a definição da arquitetura do somador onde nas linhas 11 e 12 temos a implementação das funções que geram as saídas. Agora com o código do somador completo, podemos fazer o DUT utilizando o somador completo, como componente:

```

1      library IEEE;
2      use IEEE.STD_LOGIC_1164.all;
3
4      entity dut is
5          port(A, B: in std_logic_vector (3 downto 0);
6              S: out std_logic_vector(4 downto 0)
7          );
8      end dut;
9
10     architecture dut_arch of dut is
11     component somador is
12         port(A,B,Cin: in std_logic;
13             S, Cout: out std_logic);
14     end component;
15         signal w1, w2, w3, w4: std_logic;
16
17     begin
18         INT1 : somador port map(A(0), B(0), '0', S(0), w1);
19         INT2 : somador port map(A(1), B(1), w1, S(1), w2);
20         INT3 : somador port map(A(2), B(2), w2, S(2), w3);
21         INT4 : somador port map(A(3), B(3), w3, S(3), w4);
22         S(4) <= w4;
23     end dut_arch;

```

- Listagem 2: Código do DUT

Como já descrito, declara-se, nas linhas 4 a 8 da Listagem 2, as entradas A e B como vetores de 4 bits e a saída S como vetor de 5 bits, respectivamente. Posteriormente, nas linhas 10 a 23, realiza-se a declaração da arquitetura do DUT como DUT_ARCH, e adiciona-se o component do somador completo nas linhas 11 a 14 e sinais Wn que vão os nossos fios (linha 15) e por último e feito as associações a partir da linha 17. Como visto na Figura 1, as entradas A0 e B0 são conectadas ao primeiro somador, e sua saída é conectada ao

S0, enquanto o Cin é constante nulo e Cout é enviado ao fio W1. Em seguida, as entradas A1 e B1 são devidamente conectadas, juntamente da saída S1. Contudo, agora, Cin recebe o sinal de w1, o Cout anterior, enquanto o novo Cout é enviado ao fio w2. Isso continua até o último somador, onde seu Cout, o fio w4, é conectado diretamente na saída S4. Assim, obtém-se um somador de palavras de 4 bits, ou seja, o DUT de estudo do laboratório.

2 - Questão 2

Agora vamos analisar o código do GOLDEN MODEL:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3  use IEEE.STD_LOGIC_ARITH.all;
4
5  entity GM is
6      port(A, B: in std_logic_vector (3 downto 0);
7           S: out std_logic_vector (4 downto 0)
8      );
9  end GM;
10
11 architecture GM_ARCH of GM is
12 begin
13     S <= ('0' & unsigned(A)) + ('0' & unsigned(B));
14 end GM_ARCH;
```

- Listagem 3: Código do Golden Model.

Para a funcionalidade do código, é necessário a importação da biblioteca STD_LOGIC_1164, declarada na linha 2 da Listagem 3, mas, como dito anteriormente, também importa-se STD_LOGIC_ARITH na linha 3 para a codificação do GM. Como o modelo GM é apenas uma versão garantidamente funcional do somador, suas entradas e saídas são idênticas às do DUT, como observa-se nas linhas 5–9 da Listagem 3. São declaradas duas entradas de 4 bits A e B, e uma saída de 5 bits S. Por fim, a arquitetura do GM é simples e concisa, definida nas linhas 11–14 da Listagem 3. Ocorre apenas uma declaração em diversas etapas. Primeiramente, os valores de A e B são convertidos para inteiros não-assinalados (ou seja, positivos) e, então, recebem uma concatenação de um zero à esquerda. Esse zero nada altera na soma, apenas permite que o resultado seja expresso em 5 bits ao invés de ocorrer um overflow. Por fim, soma-se A e B diretamente com o operador +, atribuindo-se tal resultado a saída S.

3 - Questão 3

O último exercício, consiste na construção do testbench que irá coordenar as entradas no DUT e GM, e checar a igualdade das saídas, segue o código do testbench:

```
1  library IEEE;
2  use IEEE.std_logic_1164.all;
3  use IEEE.std_logic_arith.all;
4  use IEEE.std_logic_unsigned.all;
5
6  entity TESTBENCH is
7      port (
8          DUT_out, GM_out: in std_logic_vector(4 downto 0);
9          A, B: out std_logic_vector(3 downto 0);
10         erro_out: out std_logic
11     );
12 end TESTBENCH;
13
14 architecture TESTBENCH_ARCH of TESTBENCH is
15 begin
16     process
17         variable currA, currB : std_logic_vector(3 downto 0);
18     begin
19         erro_out <= '0';
20         currA := "0000";
21
22         for i in 0 to 15 loop
23             A <= currA;
24             currB := "0000";
25             for j in 0 to 15 loop
26                 B <= currB;
27                 wait for 500 ns;
28
29                 if (DUT_out /= GM_out) then
30                     erro_out <= '1';
31                     assert false report "Falhou em A=" & integer'image(i) & " B=" & integer'image(j) severity ERROR;
32                 else
33                     erro_out <= '0';
34                 end if;
35
36                 currB := currB + 1;
37             end loop;
38             currA := currA + 1;
39         end loop;
40         wait;
41     end process;
42 end TESTBENCH_ARCH;
```

- Listagem 4: Código do testbench

O Testbench, circuito responsável pela análise da funcionalidade do DUT, naturalmente possui uma complexidade mais elevada se comparado aos outros circuitos presentes nesse experimento.

Além da biblioteca normal de VHDL, nota-se a presença de duas outras bibliotecas nas linhas 3 e 4 da Listagem 4: a previamente introduzida

STD_LOGIC_ARITH e uma nova, STD_LOGIC_UNSIGNED. A ARITH será utilizada para o mesmo propósito que fora anteriormente, o operador + permitirá a permutação de todas as possíveis entradas. Já a biblioteca UNSIGNED permite a fácil manipulação de vetores para realizar operações aritméticas em decimal. Em relação às suas entradas e saídas, o Testbench recebe as duas entradas atuais do DUT e GM, na linha 8 da Listagem 4, e tem como saída as próximas entradas deles, A e B respectivamente, aqui declaradas na linha 9 e a saída com o resultado do erro onde casa ocorra aparecerá 1 se não seguirá como 0.

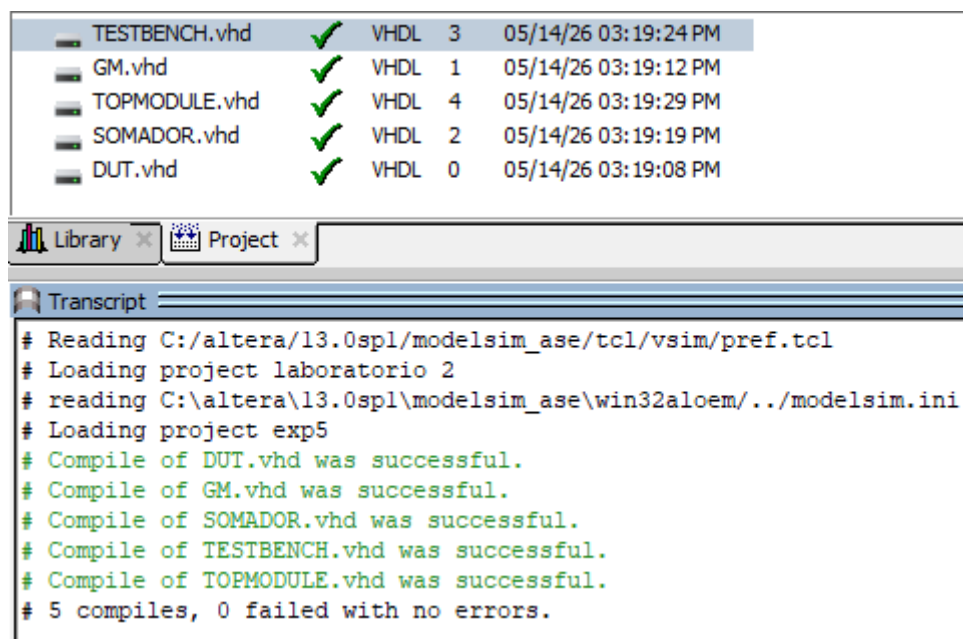
Como discutido anteriormente, para que todas as combinações de A e B sejam atingidas, será necessário o uso de um loop com um outro loop interno, a fim de realizar todas as possíveis permutações.

Primeiramente, na Listagem 6, após declarar a arquitetura, deve-se adicionar-se duas variáveis para a realização de operações. Assim, nas linhas 4 e 5 define-se uma variável para manter salvo o valor atual de A, e outra de B, respectivamente. Em seguida, começam-se os loops. A variável currA é iniciada como 0 na linha 20, e então atribua à saída A na linha 23. O mesmo é feito para currB. Então, espera-se 500ns, na linha 15, para permitir que os resultados nos circuitos do DUT e GM sejam corretamente processados.

E então é comparado o valor de A e B 29 a 34 por meio de um if **Se forem diferentes**: O sinal erro_out sobe para '1' e o comando assert imprime uma mensagem de erro no console da simulação, dizendo exatamente em quais valores de A e B o erro ocorreu. Caso não ocorra o sinal seguirá como 0, onde foi definido na linha 19.

COMPILAÇÃO

Os códigos gerados anteriormente foram submetidos a uma compilação com o intuito de garantir o seu funcionamento:



• Figura 2: Compilação dos códigos

Como visto na figura 2, nenhum código tem algum erro de sintaxe.

SIMULAÇÃO E ANÁLISE

Por fim, para a simulação foi criado um código chamado TOPMODULE, o Top Module que apenas realiza as conexões necessárias entre os três circuitos. Segue o código do mesmo:

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity TOPMODULE is
5  end TOPMODULE;
6
7  architecture TOPMODULE_ARCH of TOPMODULE is
8  component DUT is
9      port (
10         A, B: in std_logic_vector(3 downto 0);
11         S: out std_logic_vector(4 downto 0)
12     );
13 end component;
14
15 component GM is
16     port (
17         A, B: in std_logic_vector(3 downto 0);
18         S: out std_logic_vector(4 downto 0)
19     );
20 end component;
21
22 component TESTBENCH is
23     port (
24         DUT_out, GM_out: in std_logic_vector(4 downto 0);
25         A, B: out std_logic_vector(3 downto 0);
26         erro_out: out std_logic
27     );
28 end component;
29
30 signal A, B : std_logic_vector(3 downto 0);
31 signal DUT_out, GM_out : std_logic_vector(4 downto 0);
32 signal erro : std_logic;
33
34 begin
35     INT1 : DUT port map(A, B, DUT_out);
36     INT2 : GM port map(A, B, GM_out);
37     INT3 : TESTBENCH port map(DUT_out, GM_out, A, B, erro);
38
39 end TOPMODULE_ARCH;

```

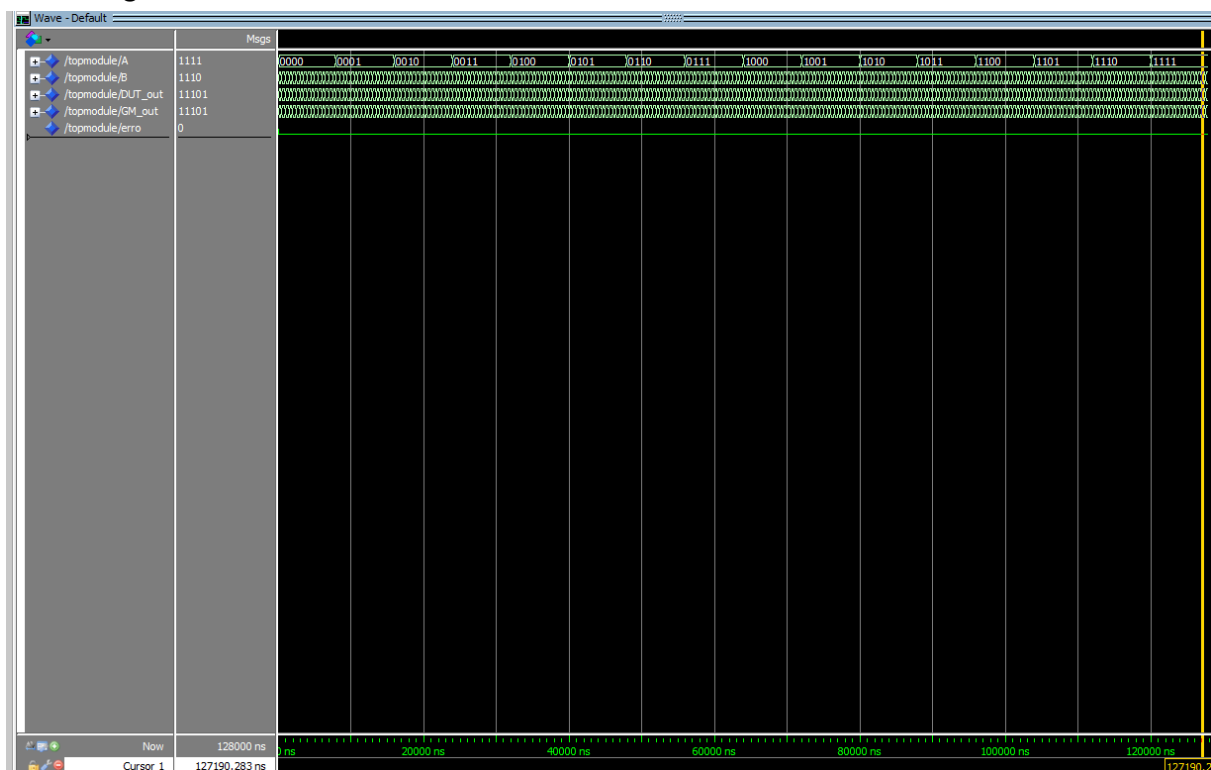
- Listagem 5: Código do topmodule.

Note que o código da listagem 5 não possui nem entradas nem saídas, uma vez que a entidade TOPMODULE é definida sem porta, nas linhas 4 e 5.

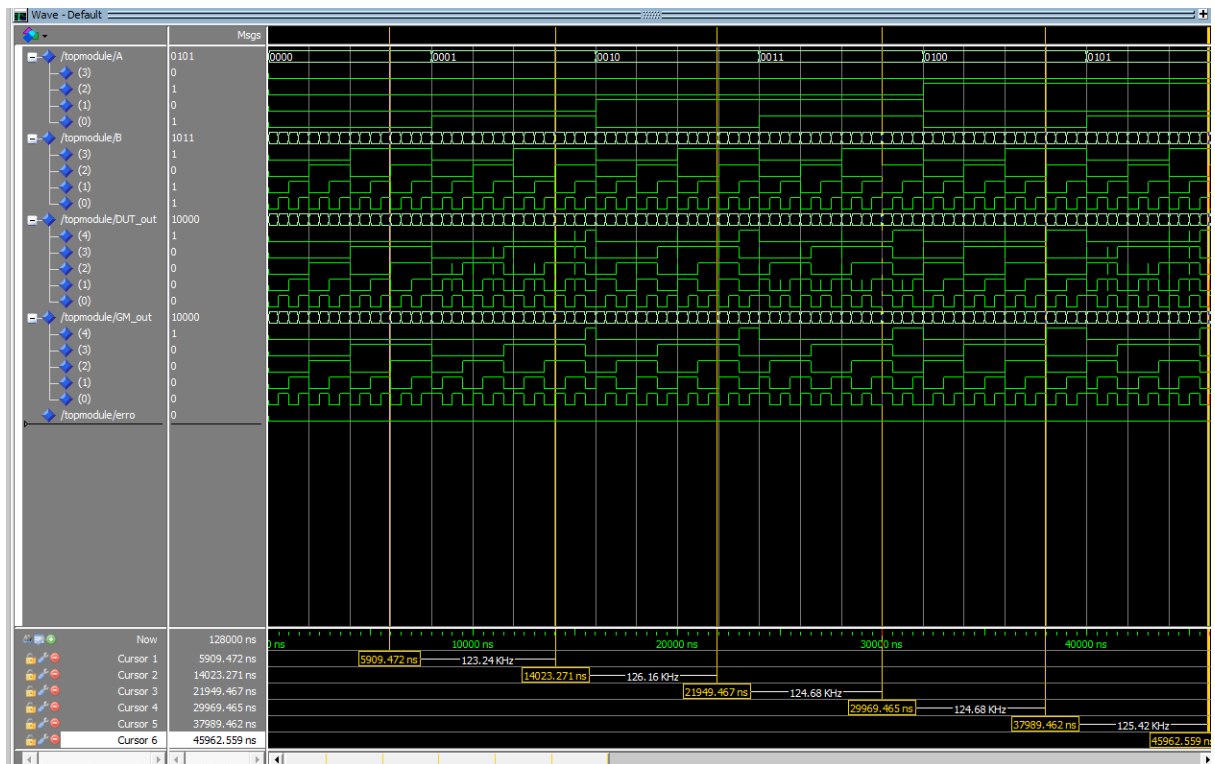
Ainda mais, sua arquitetura, representada da linha 7 a 39, consiste apenas de circuitos previamente introduzidos, os quais são conectados entre si.

Mais detalhadamente, os sinais definidos nas linhas 30 a 32 são mapeados para as entradas e saídas de mesmo nome nos componentes. Na linha 35, conecta-se A e B na entrada do DUT, enquanto coloca-se sua saída no propriamente denominado DUT_out. Faz-se similar para o GM na linha 36, apenas altera-se o nome da saída para GM_out. Já para o Testbench, na linha 37, usa-se as saídas dos outros dois componentes, DUT_out e GM_out, como entradas; inversamente, as saídas serão as entradas dos outros componentes: A e B e também é passado o sinal do erro para mostrar o mesmo.

Agora vamos simular:

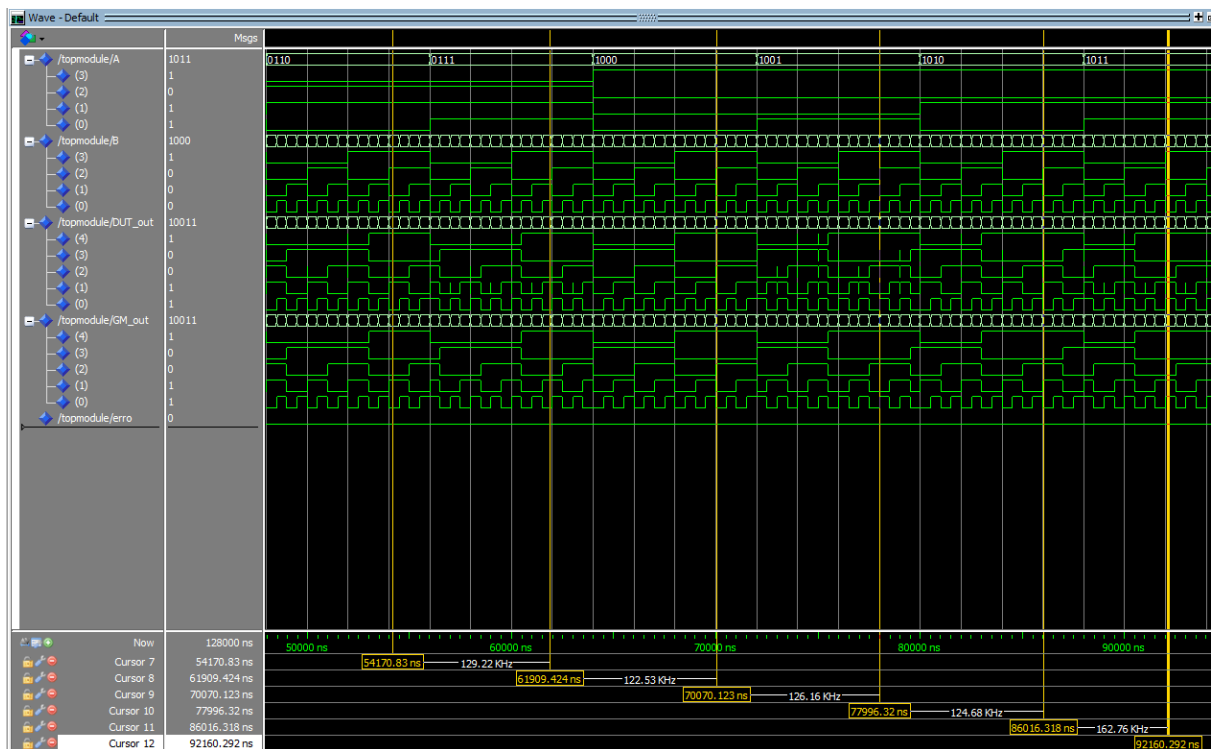


- Figura 3: Simulação com zoom completo do somador de palavras binárias em forma de onda binária



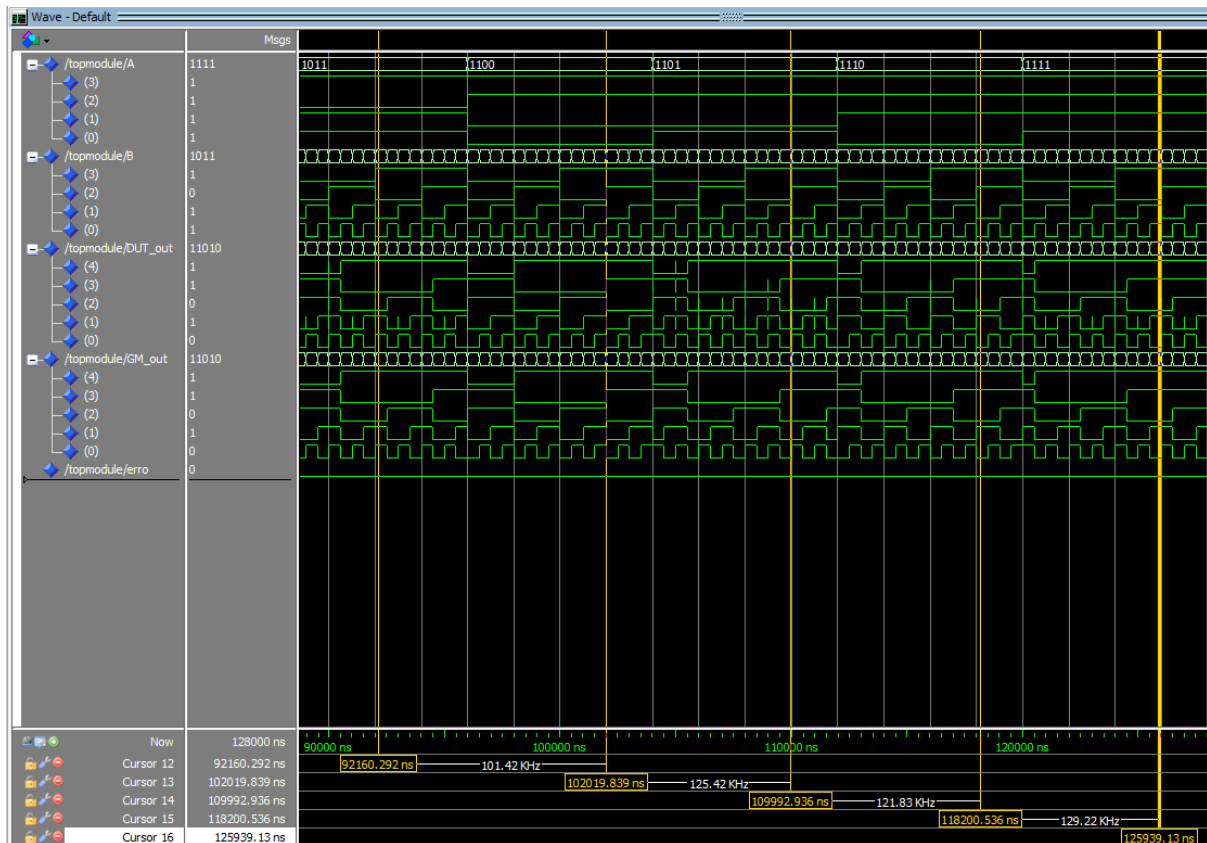
- Figura 4: simulação com A variando de 0 a 5.

Na figura 4, possuímos 5 cursores que estão em cada valor de A de 0 a 5.



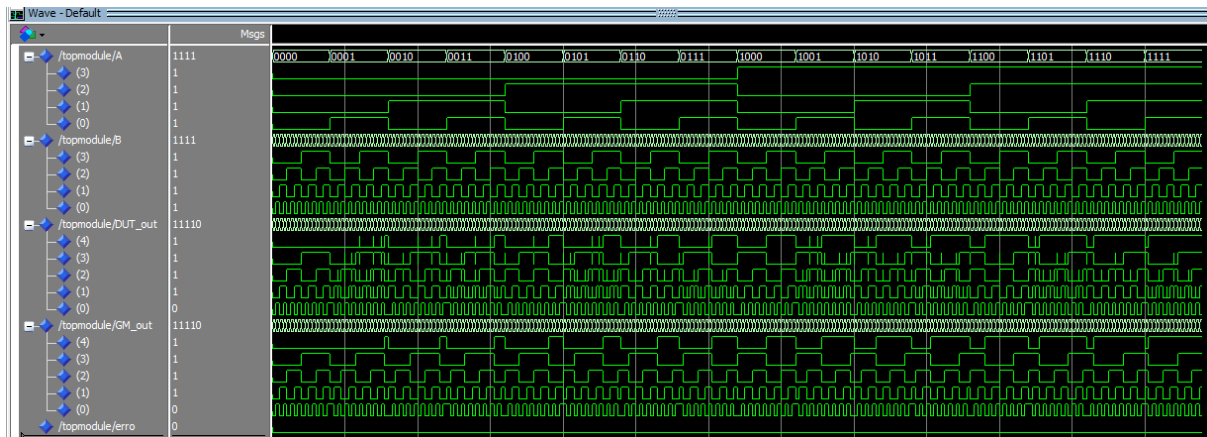
- Figura 5: Simulação com A variando de 6 a 11

Na figura 5, possuímos 6 cursores que estão em cada valor de A de 6 a 11.



- Figura 6: Simulação com A variando de 11 a 15.

Na figura 6, possuímos 5 cursores que estão em cada valor de A de 11 a 15.



- Figura 7: Simulação completa.

Como observado, a última variável chamada erro ficou toda a simulação igual a 0, como era o esperado, pois ao compararmos os códigos conseguimos ver que foram implementados da maneira correta e que apresentam as mesmas saídas.

CONCLUSÃO

Portanto, Neste experimento conseguimos com êxito implementar dois circuitos de somadores de palavras binárias, sendo um com o somador completo de 3 bits e o outro com a biblioteca STD_LOGIC_ARITH e também testá los e simulá los por meio do testbench e comparar se ambos apresentam os mesmo resultado, chegando a conclusão de que sim eles apresentam as mesmas saídas e conseguem implementar o mesmo circuitos com formas diferentes de codificação em vhdl. Não foram encontrados erros ou divergências neste experimento.